```
sentences = [
    "I am studying Machine Learning this semester.",
    "I completed the Networking course last year.",

    "Tiger is the national animal of India.",
    "India has a rich cultural heritage.",
    "Rajasthan is a famous state in India.",

    "I enjoy playing football with my friends.",
    "I like watching basketball matches."
]
```

```
import numpy as np
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer #TF-BoW
from sklearn.feature_extraction.text import TfidfVectorizer #TF-IDF
from sklearn.metrics.pairwise import cosine_similarity
```

```
#TF-BowEmbedding
tf_vectorizer=CountVectorizer()
tf_emb=tf_vectorizer.fit_transform(sentences)
```

```
#similarity matrix using cosine_similarity
tf_sim=cosine_similarity(tf_emb)
```

```
tf_sim
```

```
array([[1.        , 0.        , 0.        , 0.        , 0.        ,
        0.        , 0.        ],
       [0.        , 1.        , 0.15430335, 0.        , 0.        ,
        0.        , 0.        ],
       [0.        , 0.15430335, 1.        , 0.16903085, 0.3086067 ,
        0.        , 0.        ],
       [0.        , 0.        , 0.16903085, 1.        , 0.18257419,
        0.        , 0.        ],
       [0.        , 0.        , 0.3086067 , 0.18257419, 1.        ,
        0.        , 0.        ],
       [0.        , 0.        , 0.        , 0.        , 0.        ,
        1.        , 0.        ],
       [0.        , 0.        , 0.        , 0.        , 0.        ,
        0.        , 1.        ]])
```

```
#TF-BowEmbedding
tfidf_vectorizer=TfidfVectorizer()
tfidf_emb=tfidf_vectorizer.fit_transform(sentences)
```

```
tfidf_sim=cosine_similarity(tfidf_emb)
```

```
tfidf_sim
```

```
array([[1.        , 0.        , 0.        , 0.        , 0.        ,
        0.        , 0.        ],
       [0.        , 1.        , 0.1191192 , 0.        , 0.        ,
        0.        , 0.        ],
       [0.        , 0.1191192 , 1.        , 0.09781937, 0.21578311,
        0.        , 0.        ],
       [0.        , 0.        , 0.09781937, 1.        , 0.10410758,
        0.        , 0.        ],
       [0.        , 0.        , 0.21578311, 0.10410758, 1.        ,
        0.        , 0.        ],
       [0.        , 0.        , 0.        , 0.        , 0.        ,
        1.        , 0.        ],
       [0.        , 0.        , 0.        , 0.        , 0.        ,
        0.        , 1.        ]])
```

```
#Recommend the most matching/similar sentence
def doc_sim(tfidf_sim,sentence):
  for i in range (len(sentences)):
    sim=tfidf_sim[i].copy()
    sim[i]=-1
    top_index=np.argmax(sim)
    print(sentences[i],"->",sentences[top_index])
```

```
doc_sim(tfidf_sim,sentences)
```

```
I am studying Machine Learning this semester. -> I completed the Networking course last year.
I completed the Networking course last year. -> Tiger is the national animal of India.
Tiger is the national animal of India. -> Rajasthan is a famous state in India.
India has a rich cultural heritage. -> Rajasthan is a famous state in India.
Rajasthan is a famous state in India. -> Tiger is the national animal of India.
I enjoy playing football with my friends. -> I am studying Machine Learning this semester.
I like watching basketball matches. -> I am studying Machine Learning this semester.
```

```
#SBERT for meaning level similarities

from sentence_transformers import SentenceTransformer
sbert_model = SentenceTransformer('all-MiniLM-L6-v2')

sbert_embeddings = sbert_model.encode(sentences)
sbert_sim = cosine_similarity(sbert_embeddings)

print("SBERT Similarity Matrix:\n")
print(sbert_sim)

def sbert_doc_sim(sim_matrix, sentences):
    print("\nSBERT Semantic Recommendations:\n")
    for i in range(len(sentences)):
        sim = sim_matrix[i].copy()
        sim[i] = -1
        top_index = np.argmax(sim)
        print(sentences[i], "->", sentences[top_index])


sbert_doc_sim(sbert_sim, sentences)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set i
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
modules.json: 100%  349/349 [00:00<00:00, 30.1kB/s]
config_sentence_transformers.json: 100%  116/116 [00:00<00:00, 8.96kB/s]
README.md:  10.5k/? [00:00<00:00, 683kB/s]
sentence_bert_config.json: 100%  53.0/53.0 [00:00<00:00, 5.05kB/s]
config.json: 100%  612/612 [00:00<00:00, 43.3kB/s]
model.safetensors: 100%  90.9M/90.9M [00:01<00:00, 650MB/s]
Loading weights: 100%  103/103 [00:00<00:00, 514.59it/s, Materializing param=pooler.dense.weight]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                     | Status     |  |
------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to e
tokenizer_config.json: 100%  350/350 [00:00<00:00, 15.9kB/s]
vocab.txt:  232k/? [00:00<00:00, 5.39MB/s]
tokenizer.json:  466k/? [00:00<00:00, 12.2MB/s]
special_tokens_map.json: 100%  112/112 [00:00<00:00, 4.18kB/s]
config.json: 100%  190/190 [00:00<00:00, 13.8kB/s]
SBERT Similarity Matrix:
```